

12.3.2 The Parser applicative versus the Parser monad

Let's look at one more example. Suppose we're parsing a file of comma-separated values with two columns: *date* and *temperature*. Here's an example file:

```
1/1/2010, 25
2/1/2010, 28
3/1/2010, 42
4/1/2010, 53
...
```

If we know ahead of time the file will have the *date* and *temperature* columns in that order, we can just encode this order in the Parser we construct:

```
case class Row(date: Date, temperature: Double)

val F: Applicative[Parser] = ...
val d: Parser[Date] = ...
val temp: Parser[Double] = ...

val row: Parser[Row] = F.map2(d, temp)(Row(_, _))
val rows: Parser[List[Row]] = row.sep("\n")
```

If we don't know the order of the columns and need to extract this information from the header, then we need `flatMap`. Here's an example file where the columns happen to be in the opposite order:

```
# Temperature, Date
25, 1/1/2010
28, 2/1/2010
42, 3/1/2010
53, 4/1/2010
...
```

To parse this format, where we must dynamically choose our `Row` parser based on first parsing the header (the first line starting with #), we need `flatMap`:

```
case class Row(date: Date, temperature: Double)

val F: Monad[Parser] = ...
val d: Parser[Date] = ...
val temp: Parser[Double] = ...

val header: Parser[Parser[Row]] = ...
val rows: Parser[List[Row]] =
  F.flatMap(header) { row => row.sep("\n") }
```

Here we're parsing the header, which gives us a `Parser[Row]` as its result. We then use this parser to parse the subsequent rows. Since we don't know the order of the columns up front, we're selecting our `Row` parser *dynamically*, based on the result of parsing the header.

There are many ways to state the distinction between `Applicative` and `Monad`. Of course, the type signatures tell us all we really need to know and we can understand